

这段话的核心是：**Softmax Attention 的相似度必须先让每个 query 和每个 key 相遇后才能计算，而线性注意力要想改变计算顺序，就必须把相似度拆成“query 的函数 × key 的函数”**。先看标准 Softmax Attention：

$$o_i = \sum_{j=1}^N \frac{\exp(q_i^\top k_j)}{\sum_{l=1}^N \exp(q_i^\top k_l)} v_j$$

1. 什么叫“指数作用在 $q \cdot k$ 之后”？

注意力分数是：

$$s_{ij} = \exp(q_i^\top k_j)$$

计算顺序是：

```
q_i 与 k_j 做点积
    ↓
得到一个标量 q_i^T k_j
    ↓
对这个标量取 exp
```

因此，只有同时拿到 q_i 和 k_j ，才能计算 s_{ij} 。一般来说：

$$\exp(q_i^\top k_j) \neq \exp(q_i)^\top \exp(k_j)$$

也就是说，指数函数不能简单地分别作用在 q 和 k 上，再把结果相乘。所谓“ q 与 k 被耦合”，就是指：

必须让每一对 (q_i, k_j) 单独计算

如果有 N 个 query 和 N 个 key，就有 N^2 对组合。

2. 为什么这阻止了乘法顺序交换？

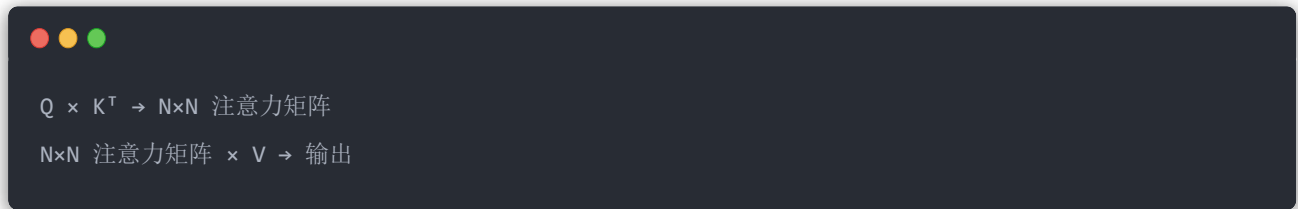
忽略 softmax 归一化，标准注意力为：

$$O = \exp(QK^\top)V$$

它的顺序是：

$$(QK^\top)V$$

计算过程为：



```

Q × K⊤ → N×N 注意力矩阵
N×N 注意力矩阵 × V → 输出
  
```

不能直接改成：

$$Q(K^\top V)$$

因为中间存在非线性：

$$\exp(QK^\top)V \neq Q(\exp(K^\top)V)$$

更准确地说， $\exp(K^\top)$ 这种写法本身也不能表达原来的逐对相似度。矩阵乘法的结合律只适用于纯乘法：

$$(QK^\top)V = Q(K^\top V)$$

加入 softmax 或指数后，结合律不能直接使用：

$$\text{softmax}(QK^\top)V \neq Q \text{softmax}(K^\top V)$$

3. 线性注意力是怎么解决的？

线性注意力重新定义相似度：

$$\text{sim}(q_i, k_j) = \phi(q_i)^\top \phi(k_j)$$

例如：

$$\phi(x) = \text{ELU}(x) + 1$$

这里的关键是，非线性被分别应用：

```
q_i → φ(q_i)
k_j → φ(k_j)
```

然后才做点积：

$$\phi(q_i)^\top \phi(k_j)$$

于是注意力输出变成：

$$o_i = \frac{\sum_j \phi(q_i)^\top \phi(k_j) v_j}{\sum_j \phi(q_i)^\top \phi(k_j)}$$

由于 $\phi(q_i)$ 与求和下标 j 无关，可以提到求和外：

$$o_i = \frac{\phi(q_i)^\top \left(\sum_j \phi(k_j) v_j^\top \right)}{\phi(q_i)^\top \left(\sum_j \phi(k_j) \right)}$$

定义固定状态：

$$S = \sum_j \phi(k_j) v_j^\top$$

以及归一化状态：

$$z = \sum_j \phi(k_j)$$

就得到：

$$o_i = \frac{\phi(q_i)^\top S}{\phi(q_i)^\top z}$$

因此可以先计算所有历史 key 和 value 的聚合：

$$S = \phi(K)^\top V$$

然后再读取：

$$O = \phi(Q)S$$

计算顺序从：

$$(\phi(Q)\phi(K)^\top)V$$

变成：

$$\phi(Q)(\phi(K)^\top V)$$

这正是“交换乘法顺序”。

4. 一个实际数字例子

假设：

$$q = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Softmax Attention

先分别计算点积：

$$q^\top k_1 = 1, \quad q^\top k_2 = 2$$

再取指数：

$$s_1 = e^1 = 2.718, \quad s_2 = e^2 = 7.389$$

归一化后：

$$a_1 = \frac{2.718}{2.718 + 7.389} = 0.269$$
$$a_2 = \frac{7.389}{2.718 + 7.389} = 0.731$$

这里的 e^1 和 e^2 ，都是在 q 与对应的 k 做完点积后才产生的。无法提前只处理 k_1 、 k_2 ，得到一个与所有 query 通用的简单聚合结果。

线性注意力

为了简化，假设：

$$\phi(x) = x + 1$$

于是：

$$\phi(q) = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$\phi(k_1) = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \phi(k_2) = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

相似度为：

$$s_1 = \phi(q)^\top \phi(k_1) = 2 \times 2 + 3 \times 1 = 7$$

$$s_2 = \phi(q)^\top \phi(k_2) = 2 \times 1 + 3 \times 2 = 8$$

归一化后：

$$a_1 = \frac{7}{15} = 0.467, \quad a_2 = \frac{8}{15} = 0.533$$

可以看到，它和 softmax 得到的：

$$[0.269, 0.731]$$

并不相同。线性注意力换取了可分解性，但也改变了相似度函数。

5. 什么叫“可分解的核”？

这里的“核”可以简单理解成计算两个向量相似度的函数：

$$K(q, k)$$

Softmax 使用的核心相似度是：

$$K_{\text{softmax}}(q, k) = \exp(q^\top k)$$

线性注意力希望把它写成：

$$K(q, k) = \phi(q)^\top \phi(k)$$

右边可以展开成：

$$K(q, k) = \sum_{r=1}^{D'} \phi_r(q) \phi_r(k)$$

这就是“可分解”：一部分只依赖 q ，另一部分只依赖 k 。一旦能够分解，就可以先聚合所有 key-value：

$$S = \sum_j \phi(k_j) v_j^\top$$

之后无论来了哪个 query，都只需查询固定状态 S 。

6. 为什么表达能力通常更受限？

Softmax 的权重是：

$$a_{ij} = \frac{\exp(q_i^\top k_j)}{\sum_l \exp(q_i^\top k_l)}$$

指数函数会放大分数差异。例如两个点积是 1 和 5：

$$e^1 \approx 2.72, \quad e^5 \approx 148.41$$

softmax 可以非常尖锐地选择第二个 key。而 ELU+1 这样的特征映射通常产生更平滑的相似度，难以完全复现 softmax 的尖锐检索模式。更根本的问题是，固定维度的 ϕ 把任意长度的历史压缩成固定状态：

$$S \in \mathbb{R}^{d_\phi \times d_v}$$

不同 token 的信息都叠加在同一个状态中。当序列非常长时，多个关联可能发生干扰。标准注意力则仍然保留每个 token 独立的 K/V，可以单独访问。

7. 原文有一处需要更严谨

“ELU+1 用可分解核近似 softmax 核” 这个说法略显宽泛。更准确地说：

- ELU+1 线性注意力通常是**用另一种可分解相似度替代 softmax 相似度**；
- Performer 等方法则明确使用随机特征，去**近似**

$$\exp(q^\top k)$$

所以应该理解为：

线性注意力为了利用结合律，需要把相似度写成 $\phi(q)^\top \phi(k)$ 。ELU+1 是一种替代 softmax 的可分解特征映射，并不一定是对 softmax 核的严格近似；这种可分解性带来了固定状态和线性复杂度，同时也限制了相似度函数及记忆容量。最简洁的总结是：

Softmax：先让 q 和 k 相遇，再做非线性

线性注意力：先分别处理 q 和 k ，再让它们相遇

前者检索能力更强，后者能够改变计算顺序、把历史压缩成固定状态。